Drifting for Reinforcement Learning: Training-Time Policy Evolution with Single-Step Inference

Bhargav Borah
Texas A&M University
bhargav_borah@tamu.edu

Abdullah Chaudhry
Texas A&M University
abdullahchaudhry@tamu.edu

Abdul Hannan Azad
Texas A&M University
hannan_123@tamu.edu

Abstract

We explore applying Drifting Models [4], a recently proposed generative modeling paradigm that evolves a single-pass generator's output distribution during training rather than inference, as a policy class for both online and offline reinforcement learning. The key appeal is that the resulting policy produces actions in a single forward pass at deployment, while retaining the expressive, multimodal character of iterative generative models such as diffusion and flow-matching policies.

*For **online RL**, we introduce **Drift-Online**: a TD3-style off-policy algorithm in which the actor is a noise-conditioned generator whose training targets are constructed via a drifting field that combines Q-gradient ascent (attraction toward high-value action regions) and RBF kernel repulsion (anti-mode-collapse pressure). A linearly decayed repulsion schedule transitions the policy from exploration to exploitation over the course of training. Evaluated on six MuJoCo continuous-control benchmarks against SAC, TD3, PPO, Gamid, PMOE, and SAC-M, Drift-Online matches or surpasses established baselines on three environments (Hopper-v4, Walker2d-v4, and Swimmer-v4) while falling short on high-dimensional tasks (Ant-v4, Humanoid-v4).*

*For **offline RL**, we introduce **Drift-Offline**: a two-phase algorithm that first pretrains a behavior-cloning (BC) anchor, then jointly trains a pessimistic Q-ensemble and a drifting actor whose attraction term is anchored to the frozen BC policy. On the OGBench antmaze-medium-navigate task, Drift-Offline reaches up to 24% success rate in early training across three seeds, but collapses in later stages without explicit decay of the repulsion term, a recurring failure mode we analyze in detail.*

*Both variants achieve **single-step (1-NFE) inference**, providing a 1.35–2.15× wall-clock speedup over K-step baselines at $K = 8$–32. Our primary findings are that drifting is a viable and conceptually appealing policy class for RL, but that both variants are sensitive to the repulsion temperature and decay schedule, a limitation we characterize and leave as an open problem for now.*

1. Introduction

Reinforcement learning (RL) requires policies that are both (i) *expressive* — able to capture complex, multimodal action distributions, and (ii) *efficient* at inference, since the policy is queried on every environment step. Diffusion [13**?**] and flow-matching [19**?**] policies deliver the former by iterating a learned denoising process at inference time, but pay a correspondingly high computational cost: generating a single action requires $K \in [8, 100]$ sequential network evaluations, which is prohibitive for real-time control. Meanwhile, Gaussian policies (as in SAC [11] or TD3 [8]) are one-step by design but struggle to model multimodal action landscapes that arise in complex tasks with ambiguous optimal behavior.

Mixture-model approaches offer a middle ground by parameterizing the policy as a Gaussian mixture [2, 5, 20], enabling multimodal coverage without iterative inference. However, these require N independent actor heads or a gating network, and do not easily scale to high-dimensional action spaces where mode separation is non-trivial. A related challenge in offline RL is distributional shift: because the dataset is fixed, policies trained via dynamic programming may query Q-values at actions outside the data support, causing value overestimation and failure at deployment [9, 16**?**].

In this work, we ask: *can we train a policy that (i) represents multimodal action distributions, (ii) requires only a single forward pass at inference, (iii) incorporates a critic signal for Q-value-guided improvement, and (iv) anchors itself to the behavioral data in the offline setting, all within a single unified framework?*

We answer this by adapting the **Drifting Models** framework [4] to reinforcement learning. In Drifting Models, a generator $f_\theta : \mathbb{R}^C \to \mathbb{R}^D$ is trained so that its pushforward distribution $q_\theta = [f_\theta]_\# p_\epsilon$ evolves toward a target p_{data} during training, rather than at inference. The mechanism is a *drifting field* $\mathbf{V}$ that attracts generated samples toward data

and repels them from each other, reaching equilibrium exactly when $q_\theta = p_{\text{data}}$. Crucially, inference remains a single forward pass regardless of how many training steps were taken.

Contributions. We make the following contributions in this paper:

1. **Drift-Online** (§3.2): a TD3-style online RL algorithm with a drifting actor. The drifting field combines Q-gradient ascent ($\mathbf{V}^+$, attraction toward high-value regions) and RBF repulsion ($\mathbf{V}^-$, anti-mode-collapse). A linearly decayed repulsion coefficient λ_t transitions the policy from exploration to exploitation. This algorithm has been evaluated on six MuJoCo tasks against six baselines.
2. **Drift-Offline** (§3.3): a two-phase offline RL algorithm with BC pretraining followed by joint critic and drifting actor training. The attraction term anchors generated actions to a frozen BC policy rather than raw dataset samples, providing a stable training-time positive distribution. This idea has been evaluated on OGBench's [18] antmaze environment.
3. **Empirical analysis**: both variants achieve 1-NFE inference (1.35–2.15× faster than K-step baselines). Drift-Online matches or surpasses SAC and TD3 on Hopper, Walker2d, and Swimmer, but fails on Ant and Humanoid. Drift-Offline collapses in late training without repulsion decay. We characterize these failure modes and identify hyperparameter sensitivity as the primary open challenge.

2. Related Work

2.1. Online Reinforcement Learning

Model-free, off-policy actor-critic methods form the dominant paradigm for continuous-action online RL. SAC [11] maximizes a maximum-entropy objective with a reparameterized Gaussian policy and automatic entropy tuning, but is suited to only problems where the action probabilities form a unimodal distribution. TD3 [8] addresses overestimation bias with clipped double Q-networks and delayed policy updates. PPO [21] is an on-policy method included as a representative alternative. These three methods serve as our primary baselines.

2.2. Multimodal Policy Representations

Gaussian mixture policies address multimodality by parameterizing the actor as a mixture of N Gaussian components. Gamid [5] combines deterministic policy gradients with a diversity loss that separates the mixture heads; SAC-M [2] extends SAC with mixture policies and a mixture entropy estimator; PMOE [20] uses a probabilistic mixture-of-experts with a gating network. These methods achieve multimodality at the cost of N actor heads, which does not naturally generalize to a single-network policy. Our approach differs: a single generator network implicitly represents a multimodal distribution via its learned noise-to-action mapping, without requiring multiple heads.

2.3. Offline Reinforcement Learning

Offline RL [**?**] learns from a fixed dataset without further environment interaction. The central challenge is distributional shift: Q-values are unreliable at out-of-distribution (OOD) actions. Solutions include explicit behavior constraints [7, 26], conservative Q-learning (CQL) [16], and in-sample regression (IQL) [15]. ReBRAC [23] revisits simple behavior-cloning-augmented TD3 as a competitive offline baseline. Our Drift-Offline algorithm falls in the behavior-constrained family, using the frozen BC policy as the behavioral anchor.

2.4. Generative Policies for Offline RL

Diffusion [13**?**] and flow-matching [17] policies model complex, multimodal action distributions but require multi-step inference. Diffuser [14] and Decision Diffuser [1] use diffusion for trajectory-level planning. FQL [19] distills a flow-matching behavior policy into a one-step surrogate. QAM [**?**] incorporates the critic gradient into flow-matching training via adjoint matching [6], achieving state-of-the-art offline RL performance, but the deployed policy still requires multi-step ODE integration. IDQL [12] uses a diffusion prior with rejection sampling at inference. Our approach avoids both distillation and iterative inference by design.

2.5. One-Step Generative Models

Consistency models [22] enforce self-consistency along denoising trajectories for one-step generation, but require careful training schedules. Shortcut models [**?**] condition the network on the desired step size. MeanFlow [10] regresses the average velocity over an integration interval. Riemannian MeanFlow [25] extends this to structured manifolds. These methods achieve one-step generation through distillation or self-consistency constraints, without a natural mechanism for incorporating a Q-function signal into training. Drifting Models [4] are conceptually distinct: the one-step property is native to the training-time evolution of the pushforward distribution, not a post-hoc approximation.

2.6. Drifting Models

Drifting Models [4] train a generator $f_\theta : \mathbb{R}^C \to \mathbb{R}^D$ by minimizing the squared norm of a drifting field $\mathbf{V}_{p,q}$ that is anti-symmetric ($\mathbf{V}_{p,q} = -\mathbf{V}_{q,p}$) and vanishes if and only if $q_\theta = p_{\text{data}}$. The field attracts generated samples toward data and repels them from each other via a normalized kernel mean-shift. Training is implemented as a stop-gradient fixed-point loss; inference is a single forward pass. Drifting Models achieve FID 1.54 on ImageNet 256×256 at 1 NFE,

state-of-the-art among single-step methods. The authors further show that a Drifting Policy replacing Diffusion Policy [3] on robotic manipulation matches 100-NFE performance at 1 NFE, demonstrating generalization beyond image domains. Our work extends this framework to value-guided RL, where the target distribution is not a fixed dataset but the Q-value-weighted optimal policy.

3. Method

3.1. Drifting Models Background

Let p_{data} be a target distribution and $f_\theta : \mathbb{R}^C \to \mathbb{R}^D$ a generator network. The generator induces a pushforward $q_\theta = [f_\theta]_\# p_\epsilon$ over $x = f_\theta(\epsilon)$, $\epsilon \sim p_\epsilon$ (e.g., $p_\epsilon = \mathcal{N}(0, I)$). Deng et al. [4] define the *drifting field*:

$$\mathbf{V}_{p,q}(x) = \mathbb{E}_{y^+\sim p}\big[\tilde{k}(x, y^+)(y^+ - x)\big] - \mathbb{E}_{y^-\sim q}\big[\tilde{k}(x, y^-)(y^- - x)\big], \tag{1}$$

where $\tilde{k}(x, y) = k(x, y)/\mathbb{E}_y[k(x, y)]$ is a normalized kernel. The field is anti-symmetric ($\mathbf{V}_{p,q} = -\mathbf{V}_{q,p}$) and satisfies $\mathbf{V}_{p,q}(x) = 0 \ \forall x \Leftrightarrow q_\theta = p_{\text{data}}$. Training minimizes the squared field norm via a stop-gradient fixed-point loss:

$$\mathcal{L}_{\text{drift}}(\theta) = \mathbb{E}_\epsilon\Big[\big\|f_\theta(\epsilon) - \text{sg}\big(f_\theta(\epsilon) + \mathbf{V}\big)\big\|^2\Big], \tag{2}$$

which moves $f_\theta(\epsilon)$ toward $f_\theta(\epsilon) + \mathbf{V}$ at each step. At convergence, $\mathbf{V} \to 0$ and inference is a single forward pass.

3.2. Drift-Online: Online RL with Drifting Actors

Setup. We maintain a generator $G_\phi(z, s)$ with $z \sim \mathcal{N}(0, I_{d_z})$ as the stochastic policy and twin target critics $\bar{Q}_1, \bar{Q}_2$ (updated via Polyak averaging at rate τ). An experience replay buffer $\mathcal{B}$ accumulates (s, a, r, s') transitions. The critic is trained with standard TD3 targets:

$$y = r + \gamma \min_j \bar{Q}_j(s', G_\phi(z', s')), \quad \mathcal{L}_Q = \mathbb{E}_{\mathcal{B}}\big[(Q_j(s, a) - y)^2\big]. \tag{3}$$

Drifting field for online RL. The target distribution in the online setting is the optimal Q-value-maximizing distribution. We construct the drifting field with two components.

Attraction ($\mathbf{V}^+$): Starting from the current generator sample $a_0 = G_\phi(z, s)$, we perform T steps of projected gradient ascent on $\min_j \bar{Q}_j(s, \cdot)$ in action space:

$$a_{t+1} = \text{clip}\big(a_t + \eta_{\text{in}} \nabla_a \min_j \bar{Q}_j(s, a)\big|_{a=a_t},\ -1, 1\big)$$
$$\mathbf{V}^+ = a_T - a_0 \tag{4}$$

Using the minimum over twin critics prevents maximization bias [8].

Repulsion ($\mathbf{V}^-$): Given K independent samples $\{a_0^{(k)} = G_\phi(z^{(k)}, s)\}_{k=1}^K$, the repulsion for sample $a_0^{(i)}$ is:

$$\mathbf{V}^-(a_0^{(i)}), = \sum_{j\neq i} \tilde{w}_{ij}(a_0^{(j)} - a_0^{(i)})$$
$$\tilde{w}_{ij} = \frac{\exp(-\|a_0^{(i)} - a_0^{(j)}\|^2/\sigma^2)}{\sum_{k\neq i}\exp(-\|a_0^{(i)} - a_0^{(k)}\|^2/\sigma^2)} \tag{5}$$

The drifting target for the generator is then:

$$a^* = \text{clip}(a_0 + \mathbf{V}^+ - \lambda_t \mathbf{V}^-,\ -1, 1), \tag{6}$$

and the generator loss is the stop-gradient MSE $\mathcal{L}_\phi = \|G_\phi(z, s) - \text{sg}[a^*]\|^2$.

Exploration-exploitation schedule. The repulsion coefficient λ_t decays linearly from λ_0 to λ_T over a fraction ρ of total training steps, then remains at λ_T:

$$\lambda_t = \max\big(\lambda_T,\ \lambda_0 - (\lambda_0 - \lambda_T) \cdot t/(\rho \cdot T_{\text{total}})\big). \tag{7}$$

High λ_t early in training encourages action diversity (exploration); low λ_t late in training allows the generator to concentrate on the best mode found (exploitation). This decay is necessary: without it, the repulsion term prevents convergence to any single mode, leading to persistent high variance in final performance.

3.3. Drift-Offline: Offline RL with Drifting Actors

Problem setup. Let $\mathcal{D} = \{(s, a, r, s')\}$ be a static offline dataset collected by behavior policy π_β. The goal is to learn a policy that maximizes expected returns while staying close to π_β to avoid OOD extrapolation. The optimal behavior-constrained policy satisfies $\pi^*(a \mid s) \propto \pi_\beta(a \mid s)\exp(Q^*(s, a)/\alpha)$.

Phase 1: BC pretraining. We first train a behavior-cloning actor $f_\psi : \mathcal{S} \to \mathcal{A}$ (a deterministic MLP) by minimizing MSE on dataset actions:

$$\mathcal{L}_{\text{BC}}(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\big[\|f_\psi(s) - a\|^2\big]. \tag{8}$$

After pretraining (200k steps), f_ψ is frozen. It serves as the source of behavioral anchor actions in Phase 2, effectively providing a stable approximation to $\pi_\beta(a \mid s)$ as a point estimate $\mu_{\text{BC}}(s) = f_\psi(s)$.

Phase 2: Drift RL. We jointly train a drifting actor $G_\phi(z, s)$ and a pessimistic Q-ensemble $\{Q_\xi^{(n)}\}_{n=1}^N$. The drifting field has three components:

$$\mathbf{V}^{\text{attract}}(x, s) = \mu_{\text{BC}}(s) - x, \tag{9}$$
$$\mathbf{V}^{\text{repulse}}(x_i, s) = \sum_{j\neq i} \tilde{w}_{ij}(x_j - x_i), \tag{10}$$
$$\mathbf{V}^Q(x, s) = \alpha_Q \nabla_a Q_{\text{pessimistic}}(s, a)\big|_{a=x}. \tag{11}$$

where $Q_{\text{pessimistic}}(s,a) = \bar{Q}(s,a) - \rho\,\sigma_Q(s,a)$ with $\bar{Q}$ and σ_Q the ensemble mean and standard deviation. The full field is $\mathbf{V} = \mathbf{V}^{\text{attract}} - \lambda_t \mathbf{V}^{\text{repulse}} + \mathbf{V}^{Q}$, with the same linear decay schedule (Eq. 7) on λ_t.

Using the frozen BC output $\mu_{\text{BC}}(s)$ as the positive sample in $\mathbf{V}^{\text{attract}}$ (rather than raw dataset actions) provides a consistent, smooth attraction target that is always available for any state s, regardless of how many dataset actions are present at that state.

The critic is trained via pessimistic ensemble TD:

$$y_{\text{target}}(s,a) = r + \gamma\, Q_{\text{pessimistic}}(s', G_\phi(z', s'))$$
$$\mathcal{L}_\xi = \mathbb{E}_{\mathcal{D}}\big[(Q_\xi^{(n)}(s,a) - y_{\text{target}})^2\big] \tag{12}$$

The actor loss is the same stop-gradient MSE as Eq. 2, with $\mathbf{V}$ as defined above. The Q-gradient $\nabla_a Q$ is computed through the target critic network with stop-gradient through the actor, and is element-wise clipped to $[-1, 1]$ for stability.

Key design choices. *Why a frozen BC anchor?* Anchoring attraction to the frozen BC policy (rather than raw dataset actions) provides a consistent, smooth attraction target that is always available for any queried state. Our empirical finding was that sampling dataset actions directly introduces variance from the finite dataset and yields only a single positive per state in each minibatch, making importance weighting degenerate.

Relationship to the original drifting framework. Adding $\mathbf{V}^{Q}$ breaks the strict anti-symmetry $\mathbf{V}_{p,q} = -\mathbf{V}_{q,p}$ of Eq. 1, since the Q-gradient has no corresponding counterpart in the reverse direction. However, $\mathbf{V}^{\text{attract}} - \mathbf{V}^{\text{repulse}}$ retains the drifting structure, and $\mathbf{V}^{Q}$ can be viewed as a perturbation that biases the equilibrium from $q_\theta = \pi_\beta$ toward $q_\theta \propto \pi_\beta \exp(Q/\alpha)$.

Cross-pollination (XPoll). The variant we evaluate incorporates cross-pollination: instead of per-sample local Q-ascent, the attraction target is a weighted combination of the local drifted sample and the batch-level best action $a^*_{\text{batch}} = \arg\max_{a \in \text{batch}} Q_{\text{pessimistic}}(s,a)$, guarded by a BC-distance penalty. A cosine annealing schedule transitions from local to global attraction over training. We denote this variant **Drift-Offline (XPoll)**.

4. Experiments

4.1. Experimental Setup

Online RL. We evaluate Drift-Online on six MuJoCo [24] continuous-control tasks: HalfCheetah-v4, Hopper-v4, Walker2d-v4, Ant-v4, Swimmer-v4, and Humanoid-v4. All methods are trained for 10^6 environment steps across 5 seeds. Baselines use CleanRL defaults; Gamid uses per-environment hyperparameters from Dey and Sharon [5] (Table 5). Drift-Online hyperparameters were tuned on HalfCheetah via Optuna and then held fixed across all environments. We report mean $\pm$ std of the final episodic return (averaged over the last 10 logged checkpoints).

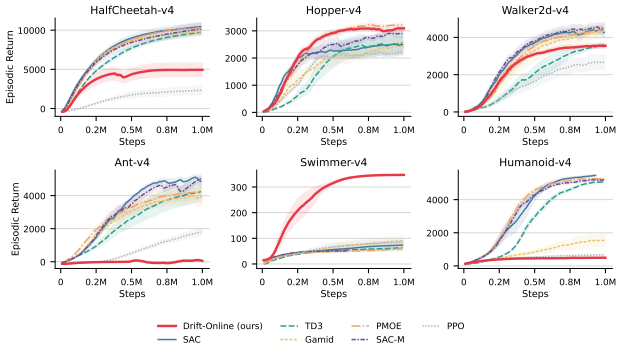

Figure 1. Learning curves on six MuJoCo tasks (mean $\pm$ 0.5 std, 5 seeds, EMA-smoothed). Drift-Online is highlighted in red.

Offline RL. We evaluate Drift-Offline (XPoll variant) on the `antmaze-medium-navigate-singletask-v0` task from OGBench [18], a goal-conditioned navigation task in which an 8-DoF ant must traverse a medium-sized maze. Three seeds are run for 200k BC pretraining steps followed by 1M drift-phase steps. Success rate is evaluated over 50 episodes every 50k steps.

Implementation. Drift-Online is implemented in PyTorch within the CleanRL [21] framework to ensure fair comparison across baselines under identical infrastructure. Drift-Offline is implemented in JAX/Flax for efficient JIT-compiled updates. All experiments were run on an NVIDIA 5090 GPU.

4.2. Online RL Results

Table 1 reports final episodic returns for all algorithms on the six MuJoCo tasks. Figure 1 shows learning curves.

Discussion. Drift-Online achieves competitive performance on three of the six tasks. On **Hopper-v4** it ranks second (3145 vs. PMOE's 3245), on **Walker2d-v4** it is competitive with TD3 (3565 vs. 3672), and on **Swimmer-v4** it is the top performer by a wide margin (347 vs. the next best Gamid's 94). The Swimmer result is particularly striking: the $K = 22$ repulsion samples appear to systematically diversify the generator across the narrow action manifold of this 2-DoF task, providing an effective exploration mechanism that neither SAC nor TD3 exploits.

On **HalfCheetah-v4**, Drift-Online underperforms substantially (4948 vs. SAC's 10553). Analysis of the drift magnitude and Q-value diagnostics (Appendix C) suggests that

Table 1. Final episodic return on MuJoCo continuous control at 1M environment steps (mean ± std, 5 seeds). **Bold**: best; underlined: second best. All methods use 1-NFE inference.

Algorithm	HalfCheetah	Hopper	Walker2d	Ant	Swimmer	Humanoid
Drift-Online (ours)	4948 ± 1689	3145 ± 138	3565 ± 230	70 ± 139	**347 ± 2**	493 ± 22
SAC	**10553 ± 1080**	2508 ± 548	4267 ± 618	4987 ± 911	74 ± 35	**5488 ± 62**
TD3	9850 ± 331	2606 ± 825	3672 ± 195	4339 ± 1180	65 ± 11	5117 ± 160
Gamid	9881 ± 709	2427 ± 629	4421 ± 628	3959 ± 1185	94 ± 38	1545 ± 742
PMOE	10357 ± 152	**3245 ± 160**	4483 ± 243	4310 ± 915	61 ± 16	5275 ± 80
SAC-M	10249 ± 1567	2896 ± 409	**4540 ± 665**	**5336 ± 135**	52 ± 7	5188 ± 117
PPO	2358 ± 1102	2257 ± 297	2702 ± 757	1898 ± 460	85 ± 24	680 ± 83

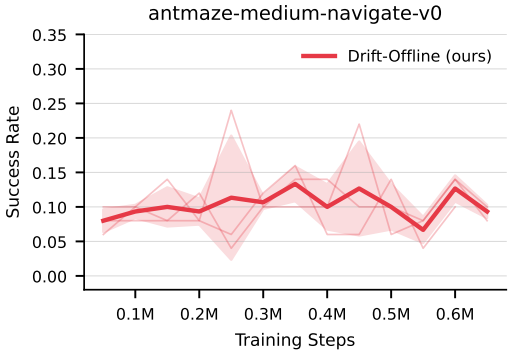

Figure 2. Success rate of Drift-Offline (XPoll) on `antmaze-medium-navigate-v0` across 3 seeds (mean ± std, thin lines show individual seeds). Training consists of 200k BC steps followed by 1M drift steps; only the drift phase is shown.

the inner-loop Q-ascent is poorly calibrated for HalfCheetah's non-smooth Q-landscape, causing the generator to track a noisy target. On **Ant-v4** and **Humanoid-v4** (8- and 17-dimensional action spaces), performance collapses to near zero. We attribute this to the curse of dimensionality in the repulsion term: in high-dimensional action spaces, all K generated actions are approximately equidistant from each other under the RBF kernel, rendering $\mathbf{V}^-$ effectively isotropic and uninformative. The attraction term $\mathbf{V}^+$ must then overcome both the uninformative repulsion and the high-dimensional Q-gradient, which we observe to be poorly conditioned in these domains.

4.3. Offline RL Results

Figure 2 shows the success rate of Drift-Offline (XPoll) across training on `antmaze-medium-navigate-v0`.

Discussion. Drift-Offline reaches a peak success rate of up to 24% in early training, demonstrating that the combination of BC anchoring and Q-gradient drift can produce meaningful navigation behavior from offline data. However, all seeds exhibit *late-training collapse*: success rate monotonically decreases after the peak. Post-hoc analysis of the training logs reveals that the critic ensemble's standard deviation σ_Q grows steadily throughout Phase 2, eventually dominating the pessimistic target and producing extremely conservative Q-values that suppress the Q-gradient signal entirely.

This collapse is exacerbated by the fixed repulsion decay schedule: without a per-run adaptive schedule, the repulsion term remains large enough in early training to maintain diversity but does not decay fast enough relative to the growing critic uncertainty. The interaction between critic calibration and repulsion decay represents the primary open challenge for Drift-Offline.

4.4. Inference Speed

Table 2. Per-action inference latency on an 5090 GPU (Drift-Offline, batch size 1). Speedup is relative to a simulated K-step sequential baseline using the same network.

Method	Latency (ms)	Speedup
Drift-Offline (1-step, ours)	0.375	1.00×
2-step sequential	0.415	0.90×
8-step sequential	0.505	0.74×
16-step sequential	0.588	0.64×
32-step sequential	0.808	0.46×

Table 2 reports per-action latency. Drift-Offline achieves 0.375 ms per action (2,667 obs/s), providing a 1.35× speedup over 8-step and 2.15× over 32-step sequential inference using the same network architecture. At batch size 256, throughput reaches 692k obs/s — relevant for massively parallel online rollout collection. The speedup values are modest at low K because GPU kernel launch overhead dominates for small networks; the advantage grows with K as expected.

5. Discussion and Conclusion

Summary. We proposed applying Drifting Models [4] to reinforcement learning as a policy class that is natively one-step at inference while retaining expressive, training-time

multimodality. We instantiated this idea in two settings: Drift-Online (TD3-style, Q-gradient ascent + RBF repulsion) and Drift-Offline (BC anchor + pessimistic critic ensemble + Q-gradient perturbation). Both achieve single-step inference, providing a 1.35–2.15$\times$ wall-clock advantage over K-step baselines at $K = 8$–32.

When does drifting work? Our results suggest that the drifting policy is most effective in lower-dimensional action spaces (Hopper, Walker2d, Swimmer) where the RBF repulsion effectively diversifies the generator over the relevant action manifold and the inner-loop Q-ascent provides a reliable attraction signal. The Swimmer result in the online setting, where Drift-Online substantially outperforms all baselines, suggests that the drifting mechanism provides a qualitatively different exploration behavior than SAC's entropy regularization or TD3's additive noise, possibly by more faithfully tracking multiple Q-value modes simultaneously.

Failure modes. Both the online and the offline variants share a common failure mode: repulsion decay is necessary but not sufficient. Without decay, the generator cannot concentrate on high-value modes, with a fixed schedule, timing the transition correctly across environments requires task-specific tuning. In high-dimensional action spaces (Ant, Humanoid), the RBF kernel becomes distance-saturated, rendering repulsion uninformative. Offline, late-stage critic variance growth dominates the Q-gradient signal, causing collapse regardless of the repulsion schedule.

Limitations and future work. The primary limitation of both variants is hyperparameter sensitivity, particularly the repulsion bandwidth σ, the decay schedule parameters $(\lambda_0, \lambda_T, \rho)$, and the inner-loop step size η_{in}. The formal convergence theory for the RL instantiations of the drifting field, which breaks strict anti-symmetry remains open. Future work should explore: (1) adaptive repulsion schedules based on critic uncertainty, (2) feature-space kernels that remain informative in high-dimensional action spaces, (3) formal analysis of the Q-gradient-perturbed drifting equilibrium, and (4) combining drifting with goal-conditioned RL for offline-to-online fine-tuning, where the one-step inference advantage is most practically significant.

6. Distribution of Responsibilities

Bhargav Borah was responsible for ideation, implementation and paper writing. Abdullah Chaudhry was responsible for implementation, and handled most of the experimentation. Abdul Hannan Azad helped with experimentation.

References

[1] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua Tenenbaum, Tommi Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? In *International Conference on Learning Representations (ICLR)*, 2023. 2

[2] Nir Baram, Guy Tennenholtz, and Shie Mannor. Maximum entropy reinforcement learning with mixture policies. *arXiv preprint arXiv:2103.10176*, 2021. 1, 2

[3] Cheng Chi, , Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. 3

[4] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026. 1, 2, 3, 5, 7

[5] Sheelabhadra Dey and Guni Sharon. Comparing deterministic and soft policy gradients for optimizing Gaussian mixture actors. *Transactions on Machine Learning Research (TMLR)*, 2024. 1, 2, 4

[6] Carles Domingo-Enrich, Michal Drozdzal, Brian Karrer, and Ricky T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations (ICLR)*, 2025. 2, 8

[7] Scott Fujimoto and Shixiang Shane Gu. A minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 2

[8] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018. 1, 2, 3

[9] Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2052–2062. PMLR, 2019. 1

[10] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025. 2

[11] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. PMLR, 2018. 1, 2

[12] Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies. *arXiv preprint arXiv:2304.10573*, 2023. 2

[13] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 1, 2

[14] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *Proceedings of the 39th International Conference on Machine Learning*. PMLR, 2022. 2

[15] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[16] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 1, 2

[17] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2023. 2

[18] Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025. 2, 4

[19] Seohong Park, Qiyang Li, and Sergey Levine. Flow Q-learning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. 1, 2

[20] Jie Ren, Yewen Li, Zihan Ding, Wei Pan, and Hao Dong. Probabilistic mixture-of-experts for efficient deep reinforcement learning. *arXiv preprint arXiv:2104.09122*, 2021. 1, 2

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 2, 4

[22] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023. 2

[23] Denis Tarasov, Vladislav Kurenkov, Alexander Nikulin, and Sergey Kolesnikov. Revisiting the minimalist approach to offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2

[24] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012. 4

[25] Dongyeop Woo, Marta Skreta, Seonghyun Park, Kirill Neklyudov, and Sungsoo Ahn. Riemannian MeanFlow. *arXiv preprint arXiv:2602.07744*, 2026. 2

[26] Yifan Wu, George Tucker, and Ofir Nachum. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361*, 2019. 2

A. Hyperparameters

Table 3. Drift-Online hyperparameters (tuned via Optuna on HalfCheetah and Walker2d, held fixed across all environments).

Hyperparameter	**Symbol**	**Value**
Architecture		
Generator hidden dims	–	$(256, 256)$
Critic hidden dims	–	$(256, 256)$
Latent noise dim	d_z	11
Optimization		
Generator learning rate	–	8.96×10^{-4}
Critic learning rate	–	5.88×10^{-4}
Batch size	B	256
Target critic Polyak	τ	4.14×10^{-3}
Discount factor	γ	0.9953
Replay buffer size	–	10^6
Learning starts	–	5000
Drifting field		
Inner-loop steps (Q-ascent)	T	3
Inner-loop step size	η_{in}	0.168
Repulsion samples	K	22
RBF kernel bandwidth	σ	0.915
Initial repulsion weight	λ_0	1.859
Final repulsion weight	λ_T	0.054
Decay fraction	ρ	0.524

Table 4. Drift-Offline (XPoll) hyperparameters.

Hyperparameter	**Symbol**	**Value**
Architecture		
Actor hidden dims	–	$(512, 512, 512, 512)$
BC actor hidden dims	–	$(512, 512, 512, 512)$
Critic hidden dims	–	$(512, 512, 512, 512)$
Critic ensemble size	N	10
Optimization		
Learning rate (all)	–	3×10^{-4}
Batch size	B	256
Target critic Polyak	τ	0.005
Discount factor	γ	0.995
Policy delay	–	every 2 critic steps
Training		
BC pretraining steps	–	2×10^5
Drift-phase steps	–	10^6
Eval episodes	–	50
Drifting field		
Repulsion samples	K	16
Repulsion mode	–	cross-batch
Kernel bandwidth	–	0.5
Q-gradient scale	α_Q	auto (XPoll)
XPoll BC penalty	–	0.5
Pessimism coefficient	ρ	0.5

B. Theoretical Motivation

We provide informal theoretical motivation for the Q-gradient drifting field, without formal proofs.

Behavior cloning recovery. When the Q-gradient scale $\alpha_Q = 0$ and $\lambda_T = 0$ (no repulsion), the field reduces to $\mathbf{V} = \mu_{\text{BC}}(s) - x$, which is satisfied at equilibrium iff $G_\phi(z, s) = \mu_{\text{BC}}(s)$ almost surely, i.e., the generator reduces to a deterministic behavior clone. When $\alpha_Q = 0$ and $\lambda_t > 0$, the attraction, repulsion balance recovers the Drifting Models objective with π_β as the target, so $q_\theta \to \pi_\beta$ at equilibrium [4].

Q-value improvement direction. The Q-gradient term $\alpha_Q \nabla_a Q(s, a)|_{a=x}$ points each generated action in the di-

rection of locally increasing Q-value. At the equilibrium of the attraction–repulsion terms (i.e., $q_\theta \approx \pi_\beta$), this perturbation biases the generator toward $\pi^* \propto \pi_\beta \exp(Q/\alpha)$, which is the optimal behavior-constrained policy. The combined fixed-point iteration thus approximates policy improvement under the behavior constraint.

Computational advantage. Because the generator is a single forward pass, the Q-gradient $\nabla_a Q(s,a)|_{a=G_\phi(z,s)}$ is computed at a single action point, requiring one backward pass through the critic. This contrasts with adjoint matching [6**?**], which requires propagating gradients along a T-step ODE trajectory at $O(T)$ cost.

Open problem. Adding $\mathbf{V}^Q$ breaks the anti-symmetry condition $\mathbf{V}_{p,q} = -\mathbf{V}_{q,p}$ of the original framework. Whether the modified field has a fixed-point and whether the stop-gradient fixed-point iteration converges to it remains an open theoretical question.

C. Training Diagnostics

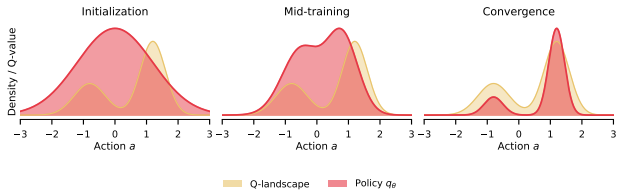

Figure 3. Illustration of drifting field evolution. The policy distribution (red) evolves from a broad initialization toward the high-Q mode (orange landscape) over training, driven by the attraction–repulsion balance.

We experimentally verified that on a 1D continuous-action bandit: with fixed repulsion, the generator faithfully models a bimodal reward landscape; with decayed repulsion, it converges on the dominant mode. This motivates the λ_t decay schedule in both Drift-Online and Drift-Offline.

For Drift-Online, training diagnostics (logged every 10k steps) include $\|\mathbf{V}^+\|$, $\|\mathbf{V}^-\|$, critic Q-values, and generator loss. The ratio $\|\mathbf{V}^+\|/\|\mathbf{V}^-\|$ serves as a proxy for the balance between exploitation and diversity, we found this ratio to be task-dependent and worth monitoring during hyperparameter search.

D. Per-Seed Online Results

The five individual seeds for Drift-Online exhibit the largest variance on HalfCheetah-v4 ($\pm$1689) and Ant-v4 ($\pm$139), indicating high sensitivity to initialization in these environments. On Swimmer-v4 the variance is essentially zero ($\pm$2), suggesting the algorithm has found a consistently effective strategy. On Hopper-v4 and Walker2d-v4, variance is comparable to TD3 ($\pm$138/$\pm$230 vs. TD3's $\pm$825/$\pm$195), indicating stable learning.